计算机网络面试八股笔记

面向 Go 后端 / AI Agent 开发实习

计算机网络面试八股笔记

一、网络模型

1. OSI 与 TCP/IP
2. 封装、分用与 MTU
3. Linux 收发网络包的路径

二、HTTP

4. 报文、方法与状态码
5. HTTP/1.0 与 HTTP/1.1
6. HTTP/1.1 的性能问题
7. HTTP/2
8. HTTP/3
9. HTTPS 与证书
10. TLS 握手
11. HTTP 缓存
12. Cookie 与 Session

三、TCP

13. TCP 提供什么
14. 三次握手
15. 为什么是三次，序列号为何要变
16. 四次挥手
17. TIME_WAIT
18. 半连接队列与全连接队列
19. 超时重传与快速重传
20. 滑动窗口与流量控制
21. 拥塞控制
22. 字节流与粘包
23. Keepalive 与 Keep-Alive
24. 崩溃、断电和拔网线

四、UDP 与 QUIC

25. UDP 的特点
26. 在 UDP 上做可靠传输
27. QUIC

五、IP

28. 地址、CIDR 与私有地址

29. 子网划分
30. ARP
31. DHCP
32. NAT
33. ICMP 与 ping
六、DNS
34. 域名的层级
35. 解析过程
36. 传输与缓存
七、从输入网址到页面显示
37. 全过程
八、WebSocket 与 RPC
38. WebSocket
39. RPC

> 使用说明：按「主题 → 知识点」组织的结构化笔记。知识框架主要参考小林coding《图解网络》（网络模型、HTTP/HTTPS、TCP、UDP/QUIC、IP、DNS、访问全过程、WebSocket 与 RPC），已重新整理改写。先记加粗关键词，再展开。协议行为以 RFC 和当前实现为准，内核参数随版本变化，面试时不要把某一版 Linux 的默认值说成永恒常数。

计算机网络面试八股笔记

一、网络模型

1. OSI 与 TCP/IP

- **OSI 七层**是教学模型：物理、数据链路、网络、传输、会话、表示、应用。会话层和表示层在实际协议栈里大多并进了应用层。
- **TCP/IP 四层**（也常说五层）：链路层、网络层（IP）、传输层（TCP/UDP）、应用层（HTTP、DNS 等）。说五层时，把链路层再拆成物理层和数据链路层。
- 对应关系：HTTP 在应用层；TCP/UDP 在传输层；IP、ICMP 在网络层；以太网、ARP 在链路层。ARP 不跑在 IP 之上，它用自己的以太网类型。
- 面试时把「这一层解决什么问题」说清楚：链路层管同一网段的帧交付，网络层管跨网络寻址和转发，传输层管端到端的进程通信，应用层管具体业务语义。

2. 封装、分用与 MTU

- 发送时自顶向下**封装**：应用数据 → TCP/UDP 段 → IP 包 → 以太网帧。接收时自底向上**分用**，靠首部里的协议号交给上一层（IP 首部的协议字段、以太网类型字段）。

- 以太网 **MTU 通常 1500 字节**。TCP 的 **MSS** 一般是 MTU 减去 IP 头和 TCP 头，IPv4 下常见 **1460**。握手时双方通告 MSS，取较小值。
- IP 层可以分片，但分片丢失要重传整个数据报，所以路径上尽量用 **Path MTU Discovery**（设置 DF，靠 ICMP「需要分片但 DF 置位」回退）。TCP 自己按 MSS 切分，避免 IP 分片。

3. Linux 收发网络包的路径

- **发送**：应用 `write`/`send` 把数据拷进 socket 发送缓冲区 → TCP 组段 → IP 选路 → 网卡驱动经 DMA 把帧交给网卡 → 网卡发出。
- **接收**：网卡 DMA 把帧写入内存（环形缓冲区）→ 硬中断只做很少的事 → **软中断**里走协议栈，校验、去头、放入 socket 接收缓冲区 → 唤醒进程 → `read` 拷到用户空间。
- CPU 不搬运网卡和内存之间的数据，搬运由 **DMA** 完成。软中断过高时，`top` 的 `si` 会升高。

二、HTTP

4. 报文、方法与状态码

- HTTP 报文是**文本**（HTTP/1）：起始行、首部、空行、正文。请求起始行是「方法、URL、版本」，响应是「版本、状态码、原因短语」。
- 常见方法：**GET** 取资源，安全且幂等；**POST** 提交，非幂等；**PUT** 整体替换，幂等；**PATCH** 部分更新；**DELETE** 删除，幂等；**HEAD** 只要首部。幂等指同一请求执行多次，服务器状态与执行一次相同，不是「没有副作用」。
- 状态码按类记：**1xx** 信息，**2xx** 成功（200、201、204），**3xx** 重定向（301 永久、302 临时、304 未修改），**4xx** 客户端错误（400、401 未认证、403 无权限、404），**5xx** 服务端（500、502 网关、503 过载）。
- 401 和 403 不同：401 是还没证明你是谁，403 是知道你是谁但拒绝。

5. HTTP/1.0 与 HTTP/1.1

- HTTP/1.0 默认**短连接**，一个请求一次 TCP 握手。`Connection: keep-alive` 才复用连接。
- HTTP/1.1 默认**持久连接**。必须带 **Host**，才能在一个 IP 上托管多个站点。支持 **chunked** 分块传输，正文长度可以不预先知道。
- **管线化（pipelining）**允许连续发出多个请求再一起收响应，但响应必须按顺序回来，一个慢响应堵住后面的，浏览器基本不用。
- 缓存相关首部、范围请求 `Range`/`206`、`100 Continue` 也是 1.1 的能力。

6. HTTP/1.1 的性能问题

- **队头阻塞**：同一连接上，前一个响应没完，后一个响应不能开始被应用使用。

- 浏览器对同一主机的并发连接有上限（Chrome 对一个源常见是 6）。握手、慢启动都要重复付。
- 首部是重复的 ASCII 文本，Cookie 一大，每次请求都带上。
- 应用层的补救：雪碧图、JS 打包、域名分片、内联小资源。这些是在协议外面绕，文件一改就要整包重下。真正要改的是协议本身。

7. HTTP/2

- 语义不变：方法、状态码、URL 还是那一套。变的是传输语法，并且仍然跑在 **TCP** 上。`http://` 与 `https://` 的样子不变，由 ALPN 在 TLS 里协商升级到 h2。
- **二进制分帧**：帧头 9 字节，含长度、类型、标志、**31 位 Stream ID**。一条 HTTP 消息（请求或响应）由同一条流上的 HEADERS 帧和 DATA 帧组成。
- **多路复用**：一个 TCP 连接上很多条流交错发送。客户端发起的流 ID 为奇数，服务端发起的为偶数。不同流可以乱序，同一条流里的帧必须按序。这解决的是 **HTTP 层**队头阻塞。
- **HPACK** 压缩首部：静态表（61 项）+ 连接上的动态表 + Huffman。同一连接上重复首部可以变成一个索引。动态表占内存，服务端会限制单连接的请求数或并发流。
- 规范里有 **服务端推送**（PUSH_PROMISE）。浏览器后来基本废弃了它，不要把它当成现在的主流优化。
- HTTP/2 仍受 **TCP 队头阻塞**：一个包丢失，内核要等这个包重传，后面已经到达的字节也不能交给应用，这条连接上的所有流一起停。

8. HTTP/3

- HTTP/3 把传输从 TCP 换成 **QUIC（基于 UDP）**。QUIC 自己做可靠、拥塞控制和加密，TLS 1.3 做进了 QUIC 握手里。
- 流在 QUIC 里是独立的：一个流丢包，只重传这个流，其他流继续。这才去掉 TCP 那种连接级队头阻塞。
- 连接用 **Connection ID** 标识，不绑死四元组。网络切换（Wi-Fi 换蜂窝）可以迁移连接，不必重新握手。
- 代价是 UDP 可能被中间盒限制，首次部署要处理回退到 HTTP/2 的路径。用户态实现协议栈，CPU 开销和调试方式也和内核 TCP 不同。

9. HTTPS 与证书

- HTTPS = HTTP + **TLS**。TLS 提供保密、完整性和服务器身份认证，可选客户端证书。
- 服务器拿出证书，里面有公钥和域名。客户端用 CA 的公钥验证证书链，并核对域名、有效期。证书解决的是「公钥确实属于这个站点」，不是加密本身。
- 只加密不够：没有身份校验就会被中间人换成自己的公钥。

10. TLS 握手

- **TLS 1.2 + RSA 密钥传输**：ClientHello / ServerHello 协商版本和套件，服务器发证书。客户端生成预主密钥，用证书里的 **RSA 公钥加密**后送出，双方派生会话密钥。服务器私钥一旦泄露，历史流量可以解密，**没有前向保密**。
- **TLS 1.2 + ECDHE**：双方各出临时椭圆曲线公钥，算出共享秘密。临时私钥用完即弃，即使证书私钥以后泄露，过去的会话密钥也推不出来，这就是**前向保密**。证书在这里用来签名，证明临时公钥是这台服务器的。
- **TLS 1.3**：去掉了静态 RSA 密钥传输，握手合并为 **1-RTT**。ClientHello 里就带上密钥共享，ServerHello 之后的握手消息已经加密。恢复会话可以用 **0-RTT** 提前发数据，但 0-RTT 数据可能被重放，只能放幂等请求。
- TLS 1.3 和 TCP 握手仍是先后关系；有的栈支持把它们叠在一起（如 TCP Fast Open 配合），不要默认说「TLS 和 TCP 总是同时握完」。

11. HTTP 缓存

- **强缓存**：在有效期内不再请求服务器。`Cache-Control: max-age=N` 优先于 `Expires`。`no-store` 是不留副本；**`no-cache` 不是不缓存**，而是使用前必须向服务器验证。
- **协商缓存**：强缓存失效后带条件请求。`ETag` / `If-None-Match` 按内容标识比较，`Last-Modified` / `If-Modified-Since` 按时间。未变则 **304**，正文不传。ETag 比时间精确，两者同时存在时 ETag 优先。
- 刷新按钮、`Cache-Control: no-cache` 请求头会跳过强缓存直接验证。CDN 和浏览器缓存是两级，`Cache-Control` 还要分清是约束浏览器还是共享缓存（`private` / `public` / `s-maxage`）。

12. Cookie 与 Session

- **Cookie** 是服务器通过 `Set-Cookie` 让浏览器存的一小段数据，之后在匹配域名、路径的请求里自动带回。属性：`Domain`、`Path`、`Expires`/`Max-Age`（没有就是会话期）、`HttpOnly`（脚本读不到）、`Secure`（只走 HTTPS）、`SameSite`（跨站是否携带，用来压低 CSRF）。
- **Session** 是服务器上的会话状态。浏览器只存一个 **Session ID**（通常放在 Cookie 里），真正的用户数据在服务端内存、Redis 或数据库。
- 区别：Cookie 数据在客户端，每次都上传，容量小（常见 4KB 量级）、可被用户改；Session 数据在服务端，更适合登录态，但要考虑多机共享和过期清理。
- 不要把密码放进 Cookie。登录态用不可预测的 Session ID，并设置 HttpOnly 和 Secure。

三、TCP

13. TCP 提供什么

- 面向连接、字节流、可靠、有序、有流量控制和拥塞控制。可靠靠序列号、确认、重传和校验和，不是「用了 TCP 应用数据就永远不会丢」：进程崩溃、缓冲区被丢、对端不读，应用仍可

能看不到数据。

- 首部关键字段：**序列号、确认号、标志位（SYN/ACK/FIN/RST）、窗口、校验和**。数据偏移指出首部长度，因为有可选项。
- SYN 和 FIN 各消耗一个序列号，纯 ACK 不消耗。确认号表示「期望收到的下一个字节」。

14. 三次握手

双方一开始是 CLOSED。服务器 `listen` 后进入 LISTEN。

1. 客户端发 **SYN**，序列号为随机的 `client_isn`，自己进入 **SYN-SENT**。这段通常不带应用数据。
2. 服务器回 **SYN+ACK**，序列号是自己的 `server_isn`，确认号是 `client_isn + 1`，进入 **SYN-RCVD**。
3. 客户端回 **ACK**，确认号是 `server_isn + 1`，进入 **ESTABLISHED**。这一段**可以带数据**。服务器收到后也进入 ESTABLISHED。

TCP Fast Open 在满足条件时允许 SYN 带数据，这是例外，不是普通握手的默认行为。

15. 为什么是三次，序列号为何要变

- **挡住历史连接**：旧的 SYN 迟到时，服务器回的确认号对不上客户端当前的初始序列号，客户端回 RST，这条过期连接建不起来。两次握手做不到这点。RFC 793 把这点当作三次握手的首要原因。
- **同步双方的初始序列号**。序号是双向的，每一向都要被对端确认。
- **避免无谓地占用资源**：两次握手时，迟到的旧 SYN 会让服务器白分配连接。
- 初始序列号不能从 0 递增给别人猜。它要随时间变化并带随机性，降低会话劫持和旧报文误入新连接的可能。

16. 四次挥手

以客户端先关为例：

1. 客户端发 **FIN**，进入 **FIN-WAIT-1**。
2. 服务器回 **ACK**，进入 **CLOSE-WAIT**。客户端收到后进入 **FIN-WAIT-2**。这时服务器还可以继续把没发完的数据送出去。
3. 服务器发完数据再发 **FIN**，进入 **LAST-ACK**。
4. 客户端回 **ACK**，进入 **TIME-WAIT**；服务器收到后进入 CLOSED。

- 挥手比握手多一次，是因为关闭方向是单向的：收到对方的 FIN 只说明对方不再发送，自己可能还有数据。
- 如果被动方没有数据要发，并且延迟确认把第二次的 ACK 和自己的 FIN 合并，抓包会看到**三次**报文。这是合并，不是另一套状态机。

17. TIME_WAIT

- **主动关闭方**在发出最后的 ACK 后进入 TIME_WAIT，等待 **2MSL**。Linux 里这段时长是 **60 秒**（内核常量，不是 `tcp_fin_timeout`；后者管的是 FIN_WAIT_2）。
- 作用有两个：最后的 ACK 丢了，对端会重传 FIN，主动方还能再确认一次；等待旧连接的报文在网络中消失，避免窜进同一四元组的新连接。
- 服务器如果主动关连接，高并发下会堆大量 TIME_WAIT。让客户端主动关、或复用连接，能减轻这个问题。
- `tcp_tw_reuse` 默认关闭。它依赖时间戳，在 NAT 后面不同主机可能撞时间戳，打开有风险，所以不要为了「消除 TIME_WAIT」就默认打开。

18. 半连接队列与全连接队列

- **半连接队列（SYN 队列）**：服务器收到 SYN、回了 SYN+ACK，还没收到第三次 ACK。这些连接处于 **SYN_RECV**。
- **全连接队列（accept 队列）**：三次握手已经完成，等应用 `accept`。LISTEN 套接字上 `ss` 的 **Recv-Q** 是当前排队数，**Send-Q** 是队列上限。
- 全连接队列上限是 **`min(somaxconn, listen 的 backlog)`**。只改其中一个不够。改完要重新 `listen`（重启进程）才生效。早期内核 `somaxconn` 默认 128，较新内核默认已提高到 4096，以 `/proc/sys/net/core/somaxconn` 为准。
- 队列满了的默认行为（`tcp_abort_on_overflow = 0`）是丢掉 ACK，客户端以为连接还在，稍后重传还能补上；设为 1 则回 RST。半连接被打满是 SYN 洪水的典型现象。
- **syncookies**：队列放不下时，把连接信息编进 SYN+ACK 的序列号，不占半连接表项，等 ACK 回来再校验。它是缓解手段，计算和选项支持都有代价。
- 半连接队列的精确公式**随内核版本变化**，不是「`tcp_max_syn_backlog` 就是长度」。排查看 `SYN_RECV` 数量和 `netstat -s` 里丢弃计数的增量。

19. 超时重传与快速重传

- 每个发出去的数据都有重传定时器。超时时间 **RTO** 由平滑 RTT 和抖动估计，并且有下限，避免在延迟抖动时重传风暴。
- 超时重传认为网络可能已经拥塞，拥塞窗口会退回去重新慢启动。
- **快速重传**：收到**三个重复 ACK**（累计四个相同确认号）就认为后面的段丢了，立刻重传，不必等 RTO。配合 **SACK**，发送方能知道后面哪些段已经到达，只补洞。
- 快速重传之后进入快速恢复，而不是一定把窗口打回 1。这点和超时不同。

20. 滑动窗口与流量控制

- **流量控制**保护接收方。接收方在 ACK 里通告 **rwnd**（自己接收缓冲区还剩多少）。发送方的发送窗口不能超过 rwnd。
- 窗口往右滑：左边是已发送且已确认，中间是已发送未确认，再往右是允许发送但还没发的。

- 接收方窗口变成 0 时，发送方停住，并定期发**窗口探测**，否则一个丢失的窗口更新会让连接永远卡死。
- 这和拥塞控制是两件事：流量控制看接收方，拥塞控制看网络。实际发送窗口是 `min(rwnd, cwnd)`。

21. 拥塞控制

目的是不要把网络打满到丢包崩溃。状态用 **cwnd**（拥塞窗口）和 **ssthresh**（慢启动阈值）。

- **慢启动**：cwnd 从较小值开始，每个 RTT 大约翻倍，是指数增长，不是「慢慢发」。
- **拥塞避免**：cwnd 超过 ssthresh 后，每个 RTT 大约只加一个 MSS，线性增长。
- **超时**：视为严重拥塞。ssthresh 降为当前 cwnd 的一半，cwnd 回到初始值，重新慢启动。
- **三个重复 ACK**：快速重传后把 ssthresh 和 cwnd 降到大约一半，然后进入**快速恢复**，而不是从 1 开始。经典 New Reno 是这个行为。
- Linux 长期默认的是 **CUBIC**（窗口按立方函数涨，对大带宽延迟积更合适），不是教科书上的 Reno。**BBR** 按带宽和最小 RTT 估计，不把丢包当成唯一信号，它是可选算法，不是默认。

22. 字节流与粘包

- TCP 没有消息边界。发送方两次 `write` 可能被合成一个段，一个大 `write` 也可能拆成多个段。接收方一次 `read` 的字节数和应用的「一条消息」对不上。
- 所谓**粘包、拆包**是应用看到的现象，不是 TCP 的一种错误模式。
- 处理办法：固定长度、分隔符、或者**长度前缀**（先读 4 字节长度，再读正文）。HTTP/1.1 用 `Content-Length` 或 chunked，就是在字节流上自己划边界。
- UDP 则保留数据报边界，一次 `recv` 对应一个数据报，但也因此会丢、会乱序。

23. Keepalive 与 Keep-Alive

- **TCP Keepalive**：传输层探活。连接长时间没有数据时发探测段，判断对端是否还在。默认间隔很长（常见 2 小时量级），用于回收死连接，不是用来做低延迟心跳。
- **HTTP Keep-Alive**：应用层复用 TCP 连接，一个连接上连续发多个 HTTP 请求。和 TCP Keepalive 不是同一个开关。
- 业务心跳如果要更快发现断线，应在应用层自己发，不要依赖默认的 TCP Keepalive 间隔。

24. 崩溃、断电和拔网线

- **进程崩溃**：操作系统关闭套接字，通常发出 FIN（或 RST），对端能走完挥手或立刻收到复位。
- **主机断电、死机**：发不出 FIN。对端只能靠自己的数据发送超时、重传耗尽或 Keepalive 才发现。只读不写的连接可能很久都以为还活着。
- **拔掉网线**：本机协议栈里的连接还在，只是包出不去。插回同一地址后，未超时的连接往往还能继续。中间的 NAT 或对端可能已经因为空闲把状态清掉，这时再发包会收到 RST。

四、UDP 与 QUIC

25. UDP 的特点

- 无连接、不重传、不保序、没有拥塞控制。首部 8 字节：源端口、目的端口、长度、校验和。
- 一次发送是一个数据报，超过路径 MTU 就交给 IP 分片，分片风险高，应用应自己控制大小。
- IPv4 里 UDP 校验和可以是 0（表示不校验），IPv6 里必须有校验和。
- 适合能容忍丢包的实时音视频、以及请求很小的查询（传统 DNS）。需要可靠时由应用补，或者改用 TCP/QUIC。

26. 在 UDP 上做可靠传输

应用要自己补上 TCP 已经提供的东西：序列号、确认、超时重传、滑动窗口、拥塞控制。
QUIC、KCP 以及一些游戏协议走的都是这条路。不做拥塞控制就在公网上猛发，会伤害其他流量，也不该叫「可靠传输方案」。

27. QUIC

- 跑在 UDP 上，把连接、流、加密和拥塞控制放在用户态。HTTP/3 是它的主要应用。
- 握手通常 **1-RTT** 就能发应用数据，恢复后可以 **0-RTT**，0-RTT 同样有重放问题。
- 多流独立，没有 TCP 的连接级队头阻塞。Connection ID 支持连接迁移。
- 加密是默认的，不只加密载荷，还盖住更多传输信息，中间盒更难改写。

五、IP

28. 地址、CIDR 与私有地址

- IPv4 地址 32 位，点分十进制。IPv6 地址 128 位。
- 早期按 A/B/C 类划分太死，现在用 **CIDR**：`地址/前缀长度`。前缀是网络号，剩下是主机号。
- 私有地址不在公网路由：**10.0.0.0/8**、**172.16.0.0/12**、**192.168.0.0/16**。环回是 **127.0.0.0/8**，链路本地 IPv4 是 169.254.0.0/16。
- 主机位全 0 是网络地址，全 1 是该网段广播地址，都不能分给主机。

29. 子网划分

- 把主机位再借几位当子网号。子网数量是 2 的借用位数次方；每个子网可用主机数是 2 的剩余主机位数次方再减 2（网络地址和广播地址）。
- 例：`192.168.1.0/24` 再分成 4 个子网，要再借 2 位，变成 **/26**。每个子网 64 个地址，可用主机 62 个。四个网络地址是 `.0`、`.64`、`.128`、`.192`。
- 路由器用最长前缀匹配选路。更具体的前缀优先于更短的默认路由。

30. ARP

- 同一链路里，IP 包要装进帧，帧需要目的 **MAC**。**ARP** 用广播问「谁是这个 IP」，对方单播回答自己的 MAC。结果缓存在 ARP 表里。
- 跨网段时，目的 MAC 是**网关**的 MAC，不是最终主机的 MAC。IP 头里的目的 IP 始终是最终主机。
- **免费 ARP**：自己宣布「这个 IP 的 MAC 是我」。用来检测地址冲突，也用来在故障切换时刷新别人的 ARP 缓存。
- ARP 没有认证，同一广播域里可以伪造应答，这是 ARP 欺骗。

31. DHCP

- 主机刚上网没有 IP，用 UDP 广播完成 **DORA**：**Discover**（客户端找服务器）→ **Offer**（服务器给一个地址）→ **Request**（客户端选定）→ **Ack**（服务器确认租约）。端口是服务器 67、客户端 68。
- 租约到期前要续租。常见实现在租期的 50%（T1）向原服务器单播续租，87.5%（T2）仍失败就重新广播。
- DHCP 还能下发掩码、网关和 DNS。它工作在还没有 IP 的阶段，所以早期报文是广播，不能靠普通单播路由穿过三层，除非有 DHCP 中继。

32. NAT

- 私有地址出公网时，路由器改写 IP 头。最常见的是 **NAPT（PAT）**：多台内网主机共用一个公网 IP，靠不同的源端口区分。这是家用路由器的默认行为。
- **SNAT** 改源地址，用于内网访问外网。**DNAT** 改目的地址，用于把公网端口映射到内网服务器。
- 连接跟踪表记下「内网地址:端口 ↔ 公网地址:端口 ↔ 对端」。表项过期后，外部再发进来就找不到主人。
- NAT 打破了端到端寻址，对端看到的不是真实源地址。P2P 要靠打洞或中继。IPv6 的一个目的就是减少对 NAT 的依赖。

33. ICMP 与 ping

- ICMP 搭在 IP 上，用来报告差错和做探测，不是应用层的 TCP/UDP。
- **ping** 发 ICMP Echo Request（类型 8），对端回 Echo Reply（类型 0）。它证明的是 IP 层能通，不证明某个 TCP 端口开着。
- **traceroute** 利用 TTL。第一跳把 TTL 设为 1，路由器减到 0 就丢弃并回 ICMP 超时（类型 11），于是知道第一跳是谁；然后 TTL 加 1，直到到达目的。
- `ping 127.0.0.1` 走的是本机环回，不经过网卡，拔了网线也能通。这只能说明本机协议栈还在。

六、DNS

34. 域名的层级

- 域名点号分隔，**越靠右层级越高**。`www.example.com.` 最右边那个点是根。根之下是顶级域（`com`），再下是权威域（`example.com`）。
- 常见记录：**A** / **AAAA** 把名字解析到 IPv4 / IPv6，**CNAME** 是别名，**NS** 授权，**MX** 邮件。
- 权威服务器持有这个域的原始数据。递归解析器替客户端跑完查询，把结果按 **TTL** 缓存。

35. 解析过程

客户端真正发出网络查询之前，通常先看浏览器缓存、操作系统缓存和本机 `hosts`。都没有才问配置里的递归 DNS：

1. 递归解析器收到 `www.example.com`。自己有缓存就直接回。
2. 没有则问**根**。根不给最终地址，只告诉 `.com` 的顶级域服务器。
3. 问顶级域，得到 `example.com` 的**权威**服务器。
4. 问权威，得到 A/AAAA 记录，缓存后再交给客户端。

- 客户端到递归解析器这一段通常是**递归查询**（「你帮我查到底」）。递归解析器到根、顶级、权威通常是**迭代查询**（「你只告诉我下一步问谁」）。
- 权威数据变更后，旧记录要等 TTL 过期，所以改 DNS 不是立刻全球生效。

36. 传输与缓存

- 传统 DNS 端口 **53**。普通查询优先 **UDP**。响应过大被截断时，标志位 **TC** 置 1，客户端改用 **TCP** 再查。
- 区域传送（AXFR）走 TCP。不要说「DNS 只用 UDP」或「区域传送全是 TCP」而不加 AXFR/IXFR 的区别。
- 加密 DNS：**DoT** 是 TLS over TCP，**DoH** 走 HTTPS，**DoQ** 走 QUIC。它们改变的是传输和隐私，名字的层级关系不变。

七、从输入网址到页面显示

37. 全过程

以打开 `https://www.example.com/index.html` 为例，中间任何一步命中缓存都可以跳过后面的网络访问。

1. **解析 URL**：取出协议、主机、端口（HTTPS 默认 443）、路径。浏览器看 HSTS，决定是否强制 TLS。
2. **缓存**：强缓存命中则直接用本地副本。否则准备带协商头的请求。

3. **DNS**：按上一节得到 IP。可能命中浏览器、操作系统、递归解析器或权威缓存。
4. **TCP**：三次握手。HTTPS 还要 **TLS 握手**，校验证书和域名，得到会话密钥。
5. **HTTP 请求**：拼好方法、路径、Host、Cookie、缓存条件，发送。服务器查资源，返回状态码、首部和正文。可能是 301/302，浏览器再请求新地址。
6. **连接复用**：HTTP/1.1 可在这条连接上继续要后续资源；HTTP/2、HTTP/3 则在同一连接上开多条流。
7. **渲染**：解析 HTML 建 DOM，解析 CSS 建 CSSOM，合成渲染树，布局，绘制。遇到 script、图片、字体等再发起新的请求。这些子资源各自再走缓存和连接复用。
8. **断开**：页面不再需要连接时才挥手。用户看到页面，不代表 TCP 已经关闭。

CDN 往往插在 DNS 和源站之间：权威 DNS 或 CNAME 把用户指到附近的边缘节点，边缘未命中才回源。

八、WebSocket 与 RPC

38. WebSocket

- HTTP 是客户端拉、服务器答，服务器不能在没有请求时主动推一帧业务数据（HTTP/2 推送也不是通用双向通道）。
- WebSocket 先走一次 HTTP **Upgrade**。客户端带 `Upgrade: websocket`、`Connection: Upgrade` 和 `Sec-WebSocket-Key`。服务器同意则返回 **101 Switching Protocols**，并回 `Sec-WebSocket-Accept`。之后同一条 TCP 连接不再按 HTTP 报文解释。
- 后续是带掩码和 opcode 的**帧**：文本、二进制、ping/pong、关闭。任意一方都可以发，是全双工。
- 它仍是一条 TCP，所以也有队头阻塞和粘包问题；帧格式自己提供了消息边界。断线要应用自己重连。

39. RPC

- **RPC** 把「调用远程函数」做成类似本地调用：客户端桩函数序列化参数，经网络发给服务端，服务端反序列化、执行、再把返回值送回来。
- 和 HTTP 资源接口的差别：HTTP 以资源、方法、状态码为中心，文本友好、容易经缓存和网关；RPC 以函数和类型为中心，通常用 **IDL**（如 protobuf）生成代码，二进制更紧凑。gRPC 跑在 HTTP/2 上，说明两者可以叠在一起，不是互斥的两层。
- 网络调用一定要单独考虑超时、重试、幂等、负载均衡和服务发现。本地函数没有这些失败模式，直接照搬重试会把非幂等操作执行多次。
- 选哪个：对外、要给浏览器和第三方用，HTTP/JSON 更合适；对内、高并发、强类型，RPC 更合适。不要为了「高级」在对外接口上只提供一个私有二进制协议。